Dataproc Serverless & Apache Iceberg

A Practical Handbook for Serverless Spark Lakehouse Tables on Google Cloud

Covers: Serverless Spark batches • Iceberg table format & ACID semantics • BigLake / Lakehouse runtime catalog • Time travel & schema evolution • A full hands-on demo you can run yourself (gcloud CLI, PySpark, and a companion notebook)

Prepared for GCP Data Engineering Training

Reflects Google Cloud naming as of July 2026 — Managed Service for Apache Spark / Lakehouse for Apache Iceberg

Table of Contents

1. What Is Dataproc Serverless?

> **IMPORTANT:** As of April–June 2026, Google Cloud has been consolidating branding: **Dataproc on Compute Engine** (clusters) and **Google Cloud Serverless for Apache Spark** (the product formerly called **Dataproc Serverless**) are now marketed together as the **Managed Service for Apache Spark**. Separately, **BigLake** was renamed **Lakehouse for Apache Iceberg**, and **BigLake metastore** is now the **Lakehouse runtime catalog** — but the underlying APIs, IAM role names, and `gcloud dataproc` / `gcloud biglake` commands are unchanged. This handbook uses the widely-recognized name 'Dataproc Serverless' throughout (matching what you'll still see in most tutorials, Stack Overflow answers, and exam guides), and notes the new names where it helps you recognize them in the console.

Dataproc Serverless lets you run Apache Spark jobs — batch or interactive — without creating, sizing, or managing a Spark/Hadoop cluster. You submit a **batch workload** (a PySpark script, a Spark SQL file, or a Java/Scala JAR), Google Cloud provisions the compute behind the scenes, runs it, autoscales while it runs, and tears everything down when it finishes. You pay only for the compute actually consumed during that run — there's no idle cluster racking up cost between jobs.

Compare this to classic Dataproc (cluster-based): you'd provision a cluster (master + worker nodes), keep it running (or carefully schedule start/stop), submit jobs to it, and manage its lifecycle yourself. Dataproc Serverless removes that entire layer of operational overhead for workloads that don't need a long-lived, shared cluster.

When Dataproc Serverless is a good fit

- Bursty or scheduled batch ETL jobs (nightly, hourly) where a cluster would sit idle most of the time.
- Teams migrating existing Spark/PySpark code to GCP who don't want to become cluster administrators.
- Workloads with unpredictable or spiky size, where autoscaling per-job beats pre-sizing a fixed cluster.
- Multi-tenant environments where different teams' jobs shouldn't compete for the same shared cluster resources.

(Section 3 of the companion handbook, Dataproc vs Dataflow Decision Framework, covers when you'd reach for Dataproc Serverless at all versus Dataflow — this handbook assumes you've already landed on Spark.)

2. What Is Apache Iceberg, and Why Pair It With Serverless Spark?

Apache Iceberg is an open **table format** for large analytic datasets stored as files (typically Parquet) in object storage like Cloud Storage. It's not a database or a query engine itself — it's a metadata layer that sits on top of your files and gives them database-like properties: atomic commits, consistent snapshots, schema evolution, and the ability to time-travel to a previous state of the table.

Before table formats like Iceberg, a 'table' in a data lake was really just 'a folder of Parquet files, by convention.' Two jobs writing to that folder at the same time could corrupt each other's output; there was no way to change a column type without rewriting everything; and there was no reliable way to ask 'what did this table look like yesterday at 9 AM?' Iceberg (and formats like it, e.g. Delta Lake, Apache Hudi) solve exactly these problems.

Why pair Iceberg specifically with Dataproc Serverless

- **One copy of data, many engines.** An Iceberg table written by a Spark batch job on Dataproc Serverless can be queried directly from BigQuery (via the BigLake / Lakehouse Iceberg connector) without exporting or copying data.
- **ACID writes from ephemeral compute.** Because Dataproc Serverless jobs are short-lived, you need a table format that makes concurrent/repeated batch writes safe without a long-running coordinator — Iceberg's snapshot-based commits are built for exactly this.
- **Cheap storage, warehouse-grade features.** You keep the low cost of Cloud Storage while getting upserts (MERGE), schema evolution, and partition evolution — features you'd otherwise need a full data warehouse for.

Typical use cases

Use Case	Why Iceberg + Serverless Spark helps
CDC / upsert pipelines	Land change-data-capture events from a source database and MERGE them into an Iceberg table incrementally, instead of rewriting the whole table every run.
Slowly changing dimensions	Track history of dimension changes using Iceberg snapshots/time travel instead of hand-rolled SCD Type 2 logic.
Shared lakehouse between Spark and BigQuery teams	Data engineers write with PySpark on Dataproc Serverless; analysts query the same physical table from BigQuery SQL — no duplicate pipeline.
Schema evolution without downtime	Add, rename, or widen columns on a huge table without rewriting historical data files.
Reproducible ML training sets	Pin a Spark or BigQuery ML training job to a specific Iceberg snapshot so results are exactly reproducible even as the table keeps changing.

3. Core Concepts & Glossary

Read this before the demo — every later step reuses these exact terms.

Term	Plain-English Definition
Dataproc Serverless (Managed Service for Apache Spark)	Google Cloud's fully managed way to run Spark workloads without provisioning a cluster. You submit a 'batch' and Google handles compute provisioning, autoscaling, and teardown.
Batch workload	A single, self-contained Spark job submission (PySpark script, Spark SQL file, or JAR) run via gcloud dataproc batches submit. Each batch is ephemeral — compute exists only for the duration of that job.
Dataproc cluster (for contrast)	The classic, long-lived alternative: a set of Compute Engine VMs (master + workers) you provision and keep running, to which you submit one or more jobs over time. Still available and sometimes the right choice (e.g. for interactive notebooks or components Serverless doesn't support).
Apache Spark	The open-source distributed data processing engine that both Dataproc clusters and Dataproc Serverless run. PySpark is its Python API.
Apache Iceberg	An open table format that adds ACID transactions, schema evolution, partition evolution, and time travel on top of data files (usually Parquet) stored in object storage.
Table format vs File format	A file format (e.g. Parquet, ORC, Avro) defines how individual files are structured internally. A table format (e.g. Iceberg, Delta Lake) defines how a *collection* of those files is organized into a single logical, transactional table — tracking which files currently belong to the table via metadata and snapshots.
Catalog (Iceberg catalog)	The service that tracks 'which metadata file currently represents the latest version of each Iceberg table' — the pointer Iceberg readers/writers consult first. On Google Cloud this is typically the BigLake metastore (Lakehouse runtime catalog).
BigLake metastore / Lakehouse runtime catalog	Google Cloud's managed Iceberg-compatible catalog service. Lets Spark, Flink, and BigQuery all resolve the same table metadata, so the same physical Iceberg table can be queried consistently from multiple engines. Renamed 'Lakehouse runtime catalog' in April 2026; the API/IAM surface still uses the name 'BigLake'.
Namespace	Iceberg's equivalent of a schema/dataset — a logical grouping of tables inside a catalog (e.g. 'sales', 'hr_demo').
Warehouse directory	The Cloud Storage path (gs://bucket/folder) where an Iceberg catalog stores the actual data and metadata files for the tables in it.
Snapshot	An immutable, point-in-time record of exactly which data files make up an Iceberg table. Every write (INSERT, MERGE, DELETE, schema change) creates a new snapshot; older snapshots are what enable time travel.

Term	Plain-English Definition
Manifest file / Manifest list	Iceberg's internal metadata files that track which data files belong to which snapshot. You generally don't touch these directly, but they're what makes Iceberg's fast metadata-only operations (like time travel and partition pruning) possible.
Time travel	Querying an Iceberg table as it existed at a previous snapshot or timestamp, e.g. SELECT * FROM table FOR SYSTEM_VERSION AS OF ... or Spark's VERSION AS OF syntax.
Schema evolution	Adding, dropping, renaming, or widening columns on an Iceberg table without rewriting existing data files — a metadata-only operation.
Partition evolution	Changing how an Iceberg table is partitioned going forward without rewriting historical data — old files keep their old partition layout, new files use the new one, and queries handle both transparently.
MERGE INTO (upsert)	A SQL statement that inserts new rows, updates matching rows, and optionally deletes rows in a single atomic operation — the standard way to apply CDC/change-data-capture events to an Iceberg table.
Compaction	Periodically rewriting many small data files into fewer, larger ones to keep query performance healthy — important because streaming/incremental writes tend to create many small files over time.
BigLake Iceberg table (BigQuery-managed vs external)	BigQuery can reference the same physical Iceberg data two ways: as a 'BigQuery-managed' Iceberg table (BigQuery controls writes) or as an 'external' Iceberg table that simply reads whatever Spark already wrote. This handbook's demo uses the external/read pattern, since Spark is the writer.
Ephemeral compute	Compute resources that exist only for the lifetime of a single job, then are torn down — the defining characteristic of Dataproc Serverless batches, as opposed to a persistent cluster.

4. Architecture: How It All Fits Together

It helps to see the full picture before running the demo. There are three moving pieces: your **Spark batch job**, the **catalog** that tracks table metadata, and the **Cloud Storage warehouse** that holds the actual data files.

1. You submit a PySpark or Spark SQL **batch workload** via gcloud dataproc batches submit.

2. Dataproc Serverless provisions ephemeral Spark compute, runs your job, and tears it down when finished.

3. Your Spark code is configured with an Iceberg catalog pointing at **BigLake metastore / Lakehouse runtime catalog** — this is where 'which file represents the current table version' is tracked.

4. Data files (Parquet) and Iceberg metadata files are written to your **warehouse directory** in Cloud Storage.

5. **BigQuery** can read the same Iceberg table directly (as an external/BigLake table) by consulting the same catalog — no data copy, no separate export/load pipeline.

> **NOTE:** The key mental model: **the catalog is the single source of truth for 'what does this table look like right now,'** and both Spark and BigQuery consult it. This is what makes 'one copy of data, queried from two engines' actually safe and consistent — without a shared catalog, Spark and BigQuery could each have a stale, disagreeing view of the table.

5. IAM Roles You Need to Know

Role (display name)	Role ID	What it lets you do
Dataproc Editor	roles/dataproc.editor	Submit and manage Dataproc Serverless batches and clusters.
Dataproc Worker	roles/dataproc.worker	Granted to the service account that actually runs your batch workload's compute.
BigQuery Data Editor	roles/bigquery.dataEditor	Needed on the Dataproc Serverless service account so it can create/update Iceberg table metadata in BigLake metastore (which is backed by BigQuery).
Storage Object Admin	roles/storage.objectAdmin	Needed on the Dataproc Serverless service account to read/write data and metadata files in the Cloud Storage warehouse directory.
BigQuery Data Viewer	roles/bigquery.dataViewer	Needed by anyone who should be able to query the resulting Iceberg table from BigQuery SQL.
BigLake Admin (context)	roles/biglake.admin	Broader administrative control over BigLake/Lakehouse catalog resources, typically reserved for platform teams setting up shared catalogs.

Rule of thumb for the demo: grant **BigQuery Data Editor** and **Storage Object Admin** to the service account your batch job runs as (by default, the Compute Engine default service account, unless you specify a custom one) — without both, the job will fail partway through with a permission error when it tries to write table metadata or data files.

6. Hands-On Demo — Step by Step

This demo submits a serverless PySpark batch job that creates an Iceberg table backed by BigLake metastore, inserts data, evolves its schema, then queries the same table directly from BigQuery. It reuses the `himanshugcpproject` project used in earlier handbooks.

> **NOTE:** Time required: ~25–35 minutes, including a few minutes of job runtime per batch.

Prerequisites

- A Google Cloud project with billing enabled (e.g. `himanshugcpproject`).
- Cloud Shell or a local machine with `gcloud` authenticated.
- Your account should have **Dataproc Editor** and enough BigQuery/Storage access to grant roles to the job's service account.
- A VPC subnet with **Private Google Access** enabled in your chosen region (required for Dataproc Serverless networking).

Set shell variables (used throughout)

```
export PROJECT_ID="himanshugcpproject"
export REGION="us-central1"
export BUCKET="himanshugcpproject-iceberg-demo"
export WAREHOUSE="gs://${BUCKET}/warehouse"
export CATALOG_NAME="demo_catalog"
export NAMESPACE="sales_ns"

gcloud config set project $PROJECT_ID
```

STEP 1 — Enable APIs and create a Cloud Storage bucket

```
gcloud services enable \
  dataproc.googleapis.com \
  bigquery.googleapis.com \
  biglake.googleapis.com

gcloud storage buckets create gs://${BUCKET} --location=${REGION}
```

STEP 2 — Grant the job's service account the roles it needs

```
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format='value(projectN
umber)')
export SERVICE_ACCOUNT="${PROJECT_NUMBER}[email protected]"

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${SERVICE_ACCOUNT}" \
  --role="roles/bigquery.dataEditor"

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${SERVICE_ACCOUNT}" \
  --role="roles/storage.objectAdmin"
```

STEP 3 — Write the PySpark job that creates an Iceberg table

Save as `create_iceberg_table.py`:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("CreateIcebergTable") \
    .config("spark.sql.catalog.demo_catalog", "org.apache.iceberg.spark.SparkCatalog"
) \
    .config("spark.sql.catalog.demo_catalog.catalog-impl",
            "org.apache.iceberg.gcp.bigquery.BigQueryMetastoreCatalog") \
    .config("spark.sql.catalog.demo_catalog.gcp_project", "PROJECT_ID") \
    .config("spark.sql.catalog.demo_catalog.gcp_location", "REGION") \
    .config("spark.sql.catalog.demo_catalog.warehouse", "WAREHOUSE") \
    .getOrCreate()

spark.sql("USE demo_catalog;")
spark.sql("CREATE NAMESPACE IF NOT EXISTS sales_ns;")
spark.sql("USE sales_ns;")
spark.sql("""
    CREATE TABLE IF NOT EXISTS orders (
      order_id INT,
      customer STRING,
      amount DOUBLE,
      status STRING
    ) USING ICEBERG
""")

spark.sql("""
    INSERT INTO orders VALUES
      (1, 'Riya Sharma', 4500.0, 'PLACED'),
      (2, 'Arjun Verma', 1200.0, 'PLACED'),
      (3, 'Meera Nair', 8900.0, 'SHIPPED')
""")

spark.sql("SELECT * FROM orders").show()
```

Before submitting, replace `PROJECT_ID`, `REGION`, *and* `WAREHOUSE` *placeholders in the script with your actual values (or template them from your shell variables).*

STEP 4 — Submit the batch job

```
gcloud dataproc batches submit pyspark create_iceberg_table.py \
  --project=${PROJECT_ID} \
  --region=${REGION} \
  --deps-bucket=${BUCKET} \
  --version=2.2 \
  --jars=gs://spark-lib/bigquery/iceberg-bigquery-catalog-1.6.1-1.0.2.jar \
  --properties=spark.jars.packages=org.apache.iceberg:iceberg-spark-runtime-3.5_2.12:
1.6.1
```

Watch progress: Console → Dataproc → Batches, or `gcloud dataproc batches list --region=${REGION}`. A successful run prints the three inserted rows in the driver output logs.

STEP 5 — Query the same table directly from BigQuery

Because both Spark and BigQuery use the same BigLake metastore catalog, the table is immediately queryable from BigQuery SQL — no export, no load job:

```
SELECT * FROM `PROJECT_ID.demo_catalog.sales_ns.orders`;
-- or, depending on how the catalog is registered as a BigQuery connection:
SELECT * FROM `PROJECT_ID`.sales_ns.orders;
```

> **NOTE:** The exact BigQuery reference syntax depends on how your BigLake catalog is registered as a connection/dataset — check BigQuery Studio's Explorer panel for the auto-discovered path if the first form doesn't resolve.

7. Bonus: Time Travel, Schema Evolution & MERGE

These three operations are why Iceberg earns its place over 'just Parquet files' — they're genuinely difficult or impossible to do safely on a plain folder of files.

Schema evolution (metadata-only, no rewrite)

```
spark.sql("ALTER TABLE sales_ns.orders ADD COLUMNS (region STRING)")
spark.sql("DESCRIBE TABLE sales_ns.orders").show()
```

MERGE INTO (upsert — the CDC pattern)

```
spark.sql("""
  MERGE INTO sales_ns.orders t
  USING (SELECT 2 AS order_id, 'SHIPPED' AS new_status) s
  ON t.order_id = s.order_id
  WHEN MATCHED THEN UPDATE SET t.status = s.new_status
  WHEN NOT MATCHED THEN INSERT (order_id, customer, amount, status)
              VALUES (s.order_id, 'unknown', 0.0, s.new_status)
""")
```

Time travel (query a previous snapshot)

```
-- List available snapshots
spark.sql("SELECT * FROM sales_ns.orders.snapshots").show(truncate=False)

-- Query the table as of a specific snapshot ID
spark.sql("SELECT * FROM sales_ns.orders VERSION AS OF 1234567890123456789").show()
```

From BigQuery, the equivalent is standard SQL's `FOR SYSTEM_TIME AS OF` temporal syntax against the same external Iceberg table, letting analysts time-travel without touching Spark at all.

8. Troubleshooting Common Issues

Symptom	Likely Cause / Fix
Batch fails with a permission error writing table metadata	The Dataproc Serverless service account is missing roles/bigquery.dataEditor (for BigLake metastore) or roles/storage.objectAdmin (for the warehouse bucket). Re-check Step 2.
“Private Google Access is not enabled” or job hangs on startup	Dataproc Serverless requires the subnet in your chosen region to have Private Google Access enabled. Enable it: gcloud compute networks subnets update SUBNET --region=REGION --enable-private-ip-google-access.
Table created by Spark isn't visible in BigQuery	Confirm both are pointed at the exact same catalog name, project, and location — a mismatch in gcp_location or gcp_project silently creates two different (dis)connected catalogs.
“ClassNotFoundException: BigQueryMetastoreCatalog”	The Iceberg-BigQuery catalog JAR wasn't included on the classpath. Double-check the --jars flag points at the correct iceberg-bigquery-catalog JAR version matching your Iceberg runtime version.
Job runs but MERGE/UPSERT is very slow	Small-file buildup from many incremental writes. Run compaction periodically (Iceberg's rewrite_data_files stored procedure) rather than letting file counts grow unbounded.
Time travel query fails with 'snapshot not found'	Snapshots expire based on your table's retention settings. If you need long-lived time travel, increase history.expire.max-snapshot-age-ms on the table, or note the snapshot ID sooner.

9. Best Practices Checklist

- Use a custom service account (not the Compute Engine default) for production Dataproc Serverless batches, scoped to only the roles that specific pipeline needs.
- Pin Iceberg and connector JAR versions explicitly in your batch submission rather than relying on defaults, so upgrades happen deliberately, not accidentally.
- Schedule regular compaction (rewrite_data_files) on tables that receive frequent small writes — left unchecked, small-file buildup degrades both Spark and BigQuery query performance.
- Set explicit snapshot retention/expiration policies rather than accepting Iceberg defaults, balancing time-travel needs against storage cost and metadata overhead.
- Design MERGE keys carefully — an unselective join condition in a MERGE INTO can silently update far more rows than intended.
- Treat the BigLake metastore/Lakehouse runtime catalog as shared infrastructure: agree on catalog/namespace naming conventions across teams before multiple pipelines start writing to it.
- For very large backfills, prefer Spark's native Iceberg writes over row-by-row MERGE — batch INSERT OVERWRITE or partition-level operations are far more efficient.

10. Companion Notebook & Cleanup

A companion notebook (`dataproc_serverless_iceberg_demo.ipynb`) accompanies this handbook. Since Dataproc Serverless batches run remotely (not inside the notebook kernel itself), the notebook's role is to **submit and monitor** the batch job via the Dataproc Python client library, write the PySpark script to a temp file automatically, then query the resulting Iceberg table from BigQuery once the job completes — giving you one place to run the whole demo end-to-end.

Cleanup

```
# Delete the Iceberg table's data/metadata
gcloud storage rm --recursive gs://${BUCKET}/warehouse

# Delete the demo bucket
gcloud storage buckets delete gs://${BUCKET}

# (Optional) remove the granted IAM roles from the service account if no longer neede
d
gcloud projects remove-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${SERVICE_ACCOUNT}" \
  --role="roles/bigquery.dataEditor"
```

11. Quick Reference & Further Reading

One-line summary of the whole workflow

```
Write PySpark/Spark SQL → submit as a Dataproc Serverless batch → Spark writes Iceb
erg
data + metadata to Cloud Storage, registering table state in BigLake metastore →
BigQuery reads the same catalog → one physical table, queried from both engines,
with ACID guarantees, schema evolution, and time travel.
```

Official documentation to bookmark

- Serverless for Apache Spark overview — docs.cloud.google.com/dataproc-serverless/docs/overview
- Create an Iceberg table with BigLake metastore — docs.cloud.google.com/dataproc-serverless/docs/quickstarts/spark-workloads-with-bigquery-metastore
- Configure BigLake metastore (Lakehouse runtime catalog) — docs.cloud.google.com/biglake/docs/configure-blms
- Use the Lakehouse runtime catalog with BigQuery tables — docs.cloud.google.com/biglake/docs/blms-use-tables
- Apache Iceberg documentation (open source) — iceberg.apache.org/docs/latest

---

End of handbook. Pair this with the companion notebook for the live demo, and the Troubleshooting table in Section 8 during Q&A.;